Programming Techniques 2026–2027

Lecture 11: The Barnes–Hut Algorithm

Hannu Parviainen

Universidad de la Laguna

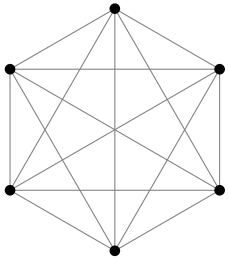
November 4, 2026

The problem: the direct sum scales as $O(N^2)$

- Every particle feels the gravity of every other particle, so the direct sum of Exercise 1 computes the force between every pair of particles.
- With N particles, there are $\frac{1}{2}N(N-1)$ pairs:

N	pairs
2	1
3	3
4	6
10	45
100	4 950
1000	499 500

- For large N, $\frac{1}{2}N(N-1) \approx \frac{1}{2}N^2$, so the run time grows as N^2. We say that the direct sum scales as $O(N^2)$.



$N = 6$: 15 pairs

Computational scaling

- If the run time T grows as N^2, ten times more particles take a hundred times longer.

	$T \propto N^2$
$T_{N=100} / T_{N=10}$	100
$T_{N=1000} / T_{N=100}$	100
$T_{N=10\,000} / T_{N=1000}$	100
$T_{N=10\,000} / T_{N=10}$	10^6

Computational scaling

- If the run time T grows as N^2, ten times more particles take a hundred times longer.
- **Not good!**

	$T \propto N^2$
$T_{N=100} / T_{N=10}$	100
$T_{N=1000} / T_{N=100}$	100
$T_{N=10\,000} / T_{N=1000}$	100
$T_{N=10\,000} / T_{N=10}$	10^6

Computational scaling

- If the run time T grows as N^2, ten times more particles take a hundred times longer.
- **Not good!**
- **Not good at all!** If $N = 1000$ takes one second, $N = 10^6$ takes 10^6 seconds, almost 12 days.

	$T \propto N^2$
$T_{N=100} / T_{N=10}$	100
$T_{N=1000} / T_{N=100}$	100
$T_{N=10\,000} / T_{N=1000}$	100
$T_{N=10\,000} / T_{N=10}$	10^6

Computational scaling

- If the run time T grows as N^2, ten times more particles take a hundred times longer.
- **Not good!**
- **Not good at all!** If $N = 1000$ takes one second, $N = 10^6$ takes 10^6 seconds, almost 12 days.
- What can we do? **The Barnes–Hut algorithm** scales as $O(N \log N)$, and $N = 10^6$ takes about half an hour.

  J. Barnes and P. Hut (1986), *A hierarchical $O(N \log N)$ force-calculation algorithm*, Nature 324, 446–449.

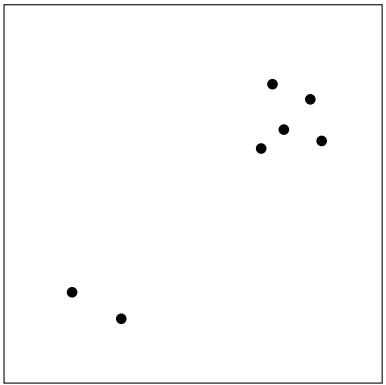
	$T \propto N^2$	$T \propto N \log N$
$T_{N=100} / T_{N=10}$	100	20
$T_{N=1000} / T_{N=100}$	100	15
$T_{N=10\,000} / T_{N=1000}$	100	13.3
$T_{N=10\,000} / T_{N=10}$	10^6	4000

How?

How?

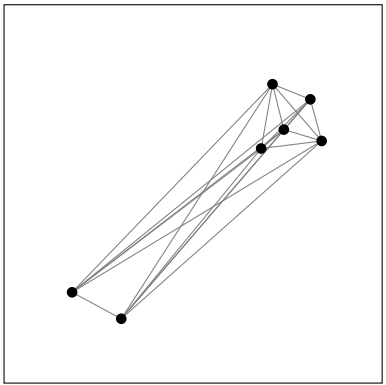
Treat a distant group of particles as a single point mass at its centre of mass.

Barnes–Hut: group the distant particles



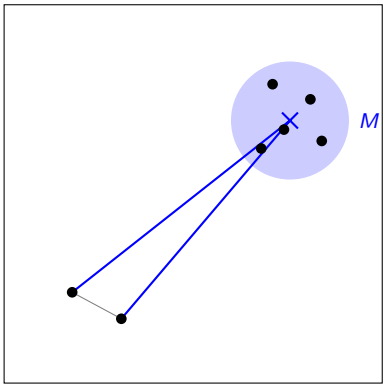
Seven particles of equal mass: a group of five, and a pair far away from it.

Barnes–Hut: group the distant particles



Direct sum: the force between every pair, $\frac{1}{2} \cdot 7 \cdot 6 = 21$ pairs. Each particle needs 6 terms.

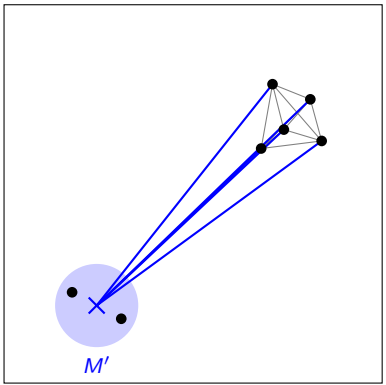
Barnes–Hut: group the distant particles



Seen from the pair, the group is far away. Its pull is almost the same as that of one particle with the group's total mass M at its centre of mass.

Each particle of the pair needs 2 terms instead of 6.

Barnes–Hut: group the distant particles

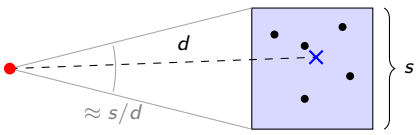


Seen from the group, the pair is far away too, and acts as one point mass M'.

The particles of the group are close to each other, so they still interact directly. Each needs 5 terms instead of 6.

With many particles, most of the others are far away, and the saving is much larger.

When is a group far enough? The opening angle θ

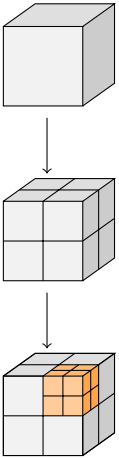


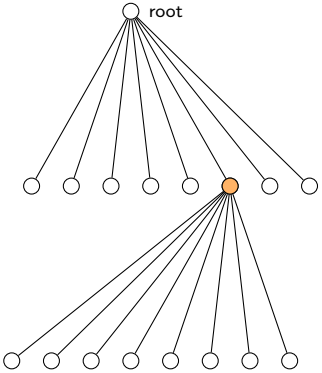
- A group inside a box of size s, whose centre of mass is at distance d, is used as a single point mass if $s/d < \theta$.
- s/d is roughly the angle at which the box is seen from the particle, so θ is called the *opening angle*.
- A smaller θ is more accurate but slower. With $\theta = 0$, no group is a point mass, and we get the direct sum.
- Typical values are $\theta \approx 0.5$–1. Exercise 2 uses $\theta = 0.5$.
- To find the groups quickly, we organise the particles in a tree.

Octrees

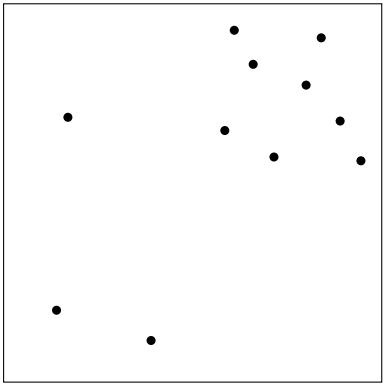
Quadtrees and octrees

- A **quadtree** divides a square into four smaller squares, and those into four again, as many times as needed.
- An **octree** does the same in 3D: it divides a cube into eight smaller cubes.
- Each node of the tree is a square (or a cube). Its children are its four quarters (or eight octants).
- The **root** contains all the particles. A **leaf** is a node without children.
- In the Barnes–Hut tree, a leaf holds at most one particle.





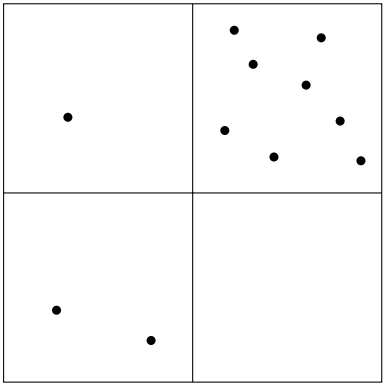
Building a quadtree



The root contains all the particles.

- The figure shows a quadtree, but an octree works in the same way.
- The root covers all the particles. Exercise 2 makes it 0.1 % larger than needed, so that no particle lies exactly on its edge.
- The figure divides the nodes level by level. The algorithm inserts the particles one at a time, but the final tree is the same whatever the order.
- Dense regions get small nodes, and empty regions stay large.
- For evenly spread particles, the depth of the tree grows as $\log N$.

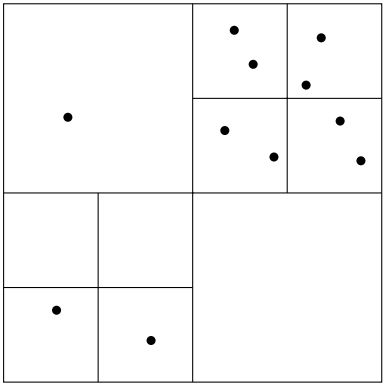
Building a quadtree



Divide every node that holds more than one particle.

- The figure shows a quadtree, but an octree works in the same way.
- The root covers all the particles. Exercise 2 makes it 0.1 % larger than needed, so that no particle lies exactly on its edge.
- The figure divides the nodes level by level. The algorithm inserts the particles one at a time, but the final tree is the same whatever the order.
- Dense regions get small nodes, and empty regions stay large.
- For evenly spread particles, the depth of the tree grows as $\log N$.

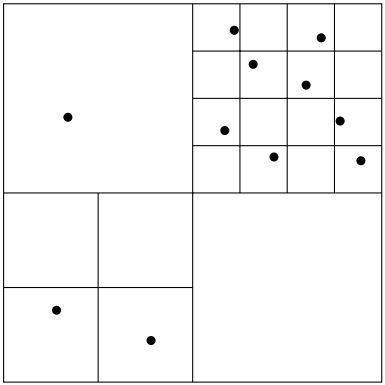
Building a quadtree



Divide every node that holds more than one particle.

- The figure shows a quadtree, but an octree works in the same way.
- The root covers all the particles. Exercise 2 makes it 0.1 % larger than needed, so that no particle lies exactly on its edge.
- The figure divides the nodes level by level. The algorithm inserts the particles one at a time, but the final tree is the same whatever the order.
- Dense regions get small nodes, and empty regions stay large.
- For evenly spread particles, the depth of the tree grows as $\log N$.

Building a quadtree



Divide again. Now every leaf holds at most one particle.

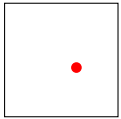
- The figure shows a quadtree, but an octree works in the same way.
- The root covers all the particles. Exercise 2 makes it 0.1 % larger than needed, so that no particle lies exactly on its edge.
- The figure divides the nodes level by level. The algorithm inserts the particles one at a time, but the final tree is the same whatever the order.
- Dense regions get small nodes, and empty regions stay large.
- For evenly spread particles, the depth of the tree grows as $\log N$.

Inserting a particle

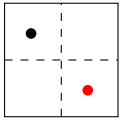
Start from the root, and go down the tree:

1. An **empty node** gets the particle.
2. A **leaf with a particle** is divided into eight children. Its old particle moves to the child that contains it, and the new particle continues as in case 3.
3. A **node with children** passes the particle on to the child that contains it.

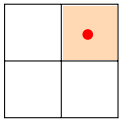
- Every node on the way adds one to its count of particles.
- Two particles at the same position would be divided forever, so the depth of the tree is limited to `max_depth`.



1. empty node

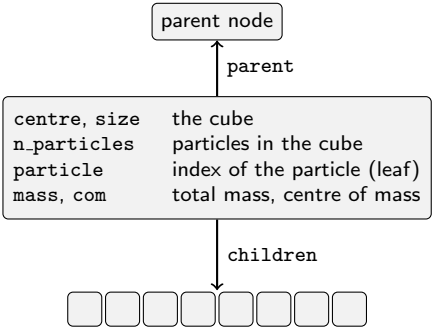


2. leaf with a particle



3. node with children

The octree node



one $2 \times 2 \times 2$ array of nodes, or null in a leaf

```
type :: node
  real(dp) :: centre(3)   ! centre of the cube
  real(dp) :: size        ! side length of the cube
  integer :: n_particles = 0
  integer :: particle = 0 ! the particle in a leaf
  real(dp) :: mass = 0.0_dp
  real(dp) :: com(3) = 0.0_dp  ! centre of mass
  type(node), pointer :: parent => null()
  type(node), pointer :: &
      children(:,:,:) => null()
end type node
```

The eight children are allocated together, as one array, when a node is divided. The particles stay in the particle arrays of the main program, and a leaf stores only the index of its particle.

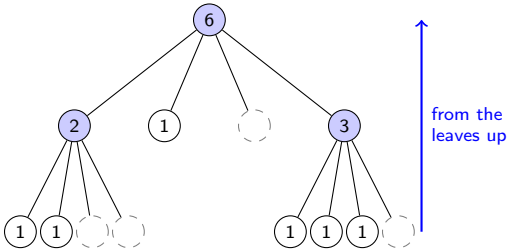
Forces

Mass and centre of mass, from the leaves up

- A leaf has the mass and position of its particle.
- A node with children gets its total mass M and centre of mass $\mathbf{R}$ from its children c:

$$M = \sum_c M_c, \qquad \mathbf{R} = \frac{1}{M} \sum_c M_c \mathbf{R}_c.$$

- A recursive procedure first calls itself for each child, and only then combines the results, so the children are always computed before their parent.
- Empty nodes have zero mass and are skipped.



masses of the nodes in a quadtree, with one unit of mass per particle

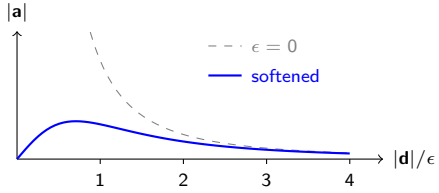
The pull of a point mass

- Single particles and point masses pull with the same formula. A mass m at the relative position $\mathbf{d}$, softened with the length ϵ, gives

$$\mathbf{a} = \frac{m\mathbf{d}}{(|\mathbf{d}|^2 + \epsilon^2)^{3/2}}.$$

- With $\epsilon = 0$, this is Newton's gravity with $G = 1$, as in Exercise 1.
- Softening keeps the pull finite when two particles pass close to each other. Exercise 2 uses $\epsilon = 0.01$.

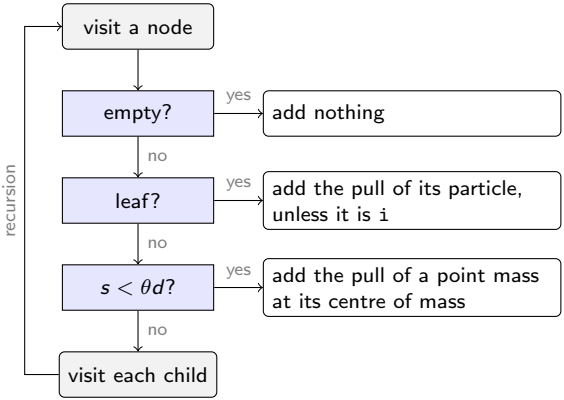
```
pure function point_mass_acceleration(mass, d) result(acc)
  real(dp), intent(in) :: mass, d(3)
  real(dp) :: acc(3)
  acc = mass * d / (sum(d**2) + eps**2)**1.5_dp
end function point_mass_acceleration
```



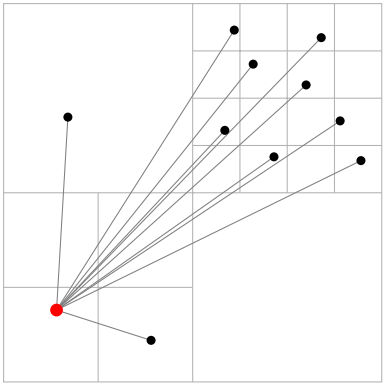
Walking the tree

The acceleration of particle `i` is a sum over a walk down the tree, starting from the root:

- An **empty node** adds nothing.
- A **leaf** adds the pull of its particle, unless it is particle `i` itself.
- A **node far enough away**, with $s < \theta d$, adds the pull of a point mass at its centre of mass.
- Any **other node** is opened: visit its children.



What one particle sees



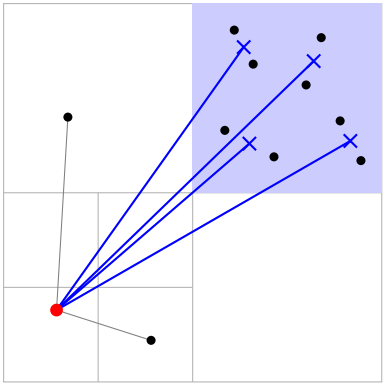
$\theta = 0$: 10 particles and 0 point masses, 10 terms in all.

The tree walk of the red particle, for three opening angles:

- Grey lines: particles used directly.
- Blue lines and shaded nodes: nodes used as point masses, with their centres of mass as crosses.

With $\theta = 0$, every node is opened, and the walk gives the direct sum.

What one particle sees



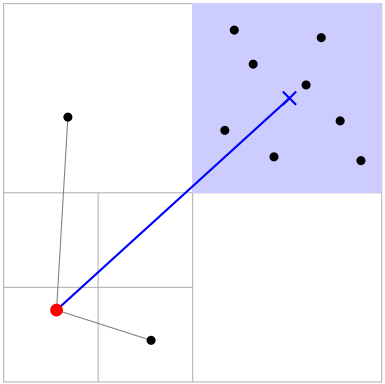
$\theta = 0.5$: 2 particles and 4 point masses, 6 terms in all.

The tree walk of the red particle, for three opening angles:

- Grey lines: particles used directly.
- Blue lines and shaded nodes: nodes used as point masses, with their centres of mass as crosses.

With $\theta = 0.5$, the distant group is replaced by the centres of mass of four nodes.

What one particle sees



$\theta = 1$: 2 particles and 1 point mass, 3 terms in all.

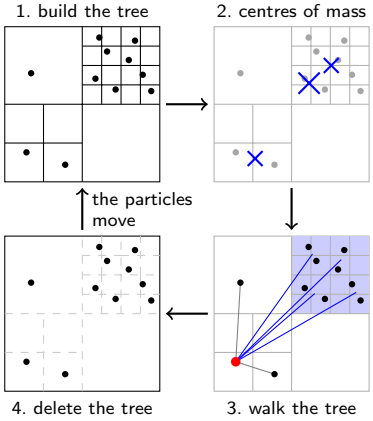
The tree walk of the red particle, for three opening angles:

- Grey lines: particles used directly.
- Blue lines and shaded nodes: nodes used as point masses, with their centres of mass as crosses.

With $\theta = 1$, the whole upper-right quarter is one point mass. The forces are faster to compute, but less accurate.

One time step

- The particles move, so the tree is built again after every drift of the leapfrog step.
- The tree is deleted after use. Otherwise every step would leak the memory of all its nodes.
- `delete_children` deallocates the children of a node after deleting their own children, again by recursion.
- Building the tree takes $O(N \log N)$ time, and each walk visits $O(\log N)$ nodes, so a step takes $O(N \log N)$ time.



Summary

- The direct sum scales as $O(N^2)$. The Barnes–Hut algorithm scales as $O(N \log N)$.
- It puts the particles into an octree, and treats a node as a single point mass when it is seen at an angle smaller than θ.
- Each time step:
  1. build the tree,
  2. compute the masses and centres of mass from the leaves up,
  3. walk the tree for each particle,
  4. delete the tree.
- Recursion fits trees well: inserting, computing the masses, walking, and deleting are all short recursive procedures.

Next: Exercise 2 starts from `lectures/exercise_2/barnes_hut.f90`, which implements everything in this lecture. The walk of each particle is independent of the others, so in the exercise you share the particles between MPI processes.